

ROS 2 使用教程

面向 Unitree Go2 RSSI 机器狗项目的对照学习手册

Go2 RSSI / ROS 2 Integration Notes

2026-06-15

目录

阅读方式	3
第一章 总体视角：ROS 2 在本项目里解决什么问题	4
1.1 一句话理解 ROS 2	4
1.2 本项目的数据和控制闭环	4
1.3 本项目当前 ROS 2 文件总表	5
第二章 ROS 2 核心概念：用本项目来理解	7
2.1 Workspace：工作空间	7
2.2 Package：功能包	7
2.3 Node：节点	8
2.4 Topic 和 Message：话题与消息	9
2.5 Publisher 和 Subscriber：发布与订阅	10
2.6 Parameter：参数	11
2.7 Launch：一键启动多个节点	11
2.8 Remapping：话题重映射	12
2.9 TF：坐标变换	12
2.10 QoS：通信质量配置	13
2.11 Action：长任务接口	13
2.12 Lifecycle：Nav2 管理节点状态	14
第三章 本项目 ROS 2 Package 逐文件对照	15
3.1 package.xml：告诉 ROS 2 这个包是什么	15
3.2 CMakeLists.txt：告诉 colcon 怎么编译	15
3.3 Unitree SDK2 链接	16
第四章 Go2 Bridge：本项目最关键的 ROS 2 节点	17
4.1 Bridge 的输入输出	17
4.2 速度命令如何进入 Go2	17
4.3 watchdog：为什么机器人会自动停	18
4.4 odom 是怎么来的	18
4.5 /go2/path_s 是怎么来的	19
4.6 IMU 是怎么发布的	19
4.7 点云处理：外参、过滤和新鲜度检查	19

第五章 直线控制器与 Nav2 客户端	21
5.1 直线控制器：最小闭环	21
5.2 Nav2 客户端：把目标交给 Nav2	21
第六章 Launch 与参数文件：本项目怎样一键启动	23
6.1 Launch 参数	23
6.2 minimal 参数文件	23
6.3 straight/obstacle 参数文件	23
6.4 采集脚本如何编排 ROS 2	24
第七章 构建、运行和调试	25
7.1 从零构建	25
7.2 只启动 bridge	25
7.3 手动发速度命令	26
7.4 直线采集推荐路径	26
7.5 Nav2 感知链路诊断	26
7.6 常用排查命令	26
第八章 ROS 2 和 RSSI/IMU 融合算法的关系	28
8.1 ROS 2 不直接等于融合算法	28
8.2 一条数据怎样进入训练或评估	28
第九章 常见问题	29
9.1 ros2 run 找不到节点	29
9.2 /odom 和 /go2/imu 没有数据	29
9.3 Nav2 active 了但机器人不动	29
9.4 点云导致机器人一直停	30
9.5 什么时候用 direct，什么时候用 Nav2	30
第十章 学习路线	31

阅读方式

这份教程不是泛泛介绍 ROS 2，而是围绕本项目已经存在的代码来讲。项目当前的 ROS 2 核心包是 `go2_nav2_bridge`，它把 Unitree Go2 的 DDS 数据转换成标准 ROS 2 话题，并把 ROS 2 的速度命令转换成 `SportClient::Move`。因此本文的主线是：

先理解 ROS 2 的基本通信模型，再把每个概念对应到本项目的 Go2 bridge、RSSI/IMU 采集、Nav2 后端和直线控制器。

建议按下面顺序读：

1. 第 [一](#) 章先建立 ROS 2 在本项目里的总体图。
2. 第 [二](#) 章学习 node、topic、message、parameter、launch、TF、QoS、action 等基础概念。
3. 第 [三](#) 章对照 `go2_nav2_bridge` 的文件结构和 CMake/package 配置。
4. 第 [四](#) 章细看 Go2 bridge 怎样发布 `/odom`、`/go2/imu`、`/go2/path_s`、`/go2/pointcloud`，以及怎样接收 `/cmd_vel`。
5. 第 [七](#) 章学习如何构建、运行、看话题、查 TF、查参数和定位问题。

本文约定

本文中的“本项目”指当前仓库中的 Go2 RSSI 路径进度项目。ROS 2 主要运行在外部电脑上，Go2 侧通过 Unitree SDK2/DDS 提供真实运动状态、IMU、点云和运动控制接口。RSSI/IMU 融合算法的主输出是路径进度 s ，ROS 2 的任务是把机器人运动、传感器、采集脚本和可选 Nav2 后端组织起来。

第一章 总体视角：ROS 2 在本项目里解决什么问题

1.1 一句话理解 ROS 2

ROS 2 可以先简单理解成“机器人软件的通信和组织框架”。它不直接等于控制算法，也不直接等于导航算法，而是负责让很多独立程序可靠地互相通信：

- Go2 bridge 负责和 Unitree SDK2/DDS 打交道。
- 直线控制器或 Nav2 负责产生速度命令。
- RSSI/IMU 采集程序负责保存实验数据。
- RViz、命令行工具、诊断脚本负责观察系统状态。

这些程序不需要写在同一个进程里。它们通过 ROS 2 的 topic、service、action、parameter、TF 等机制连接。

1.2 本项目的数据和控制闭环

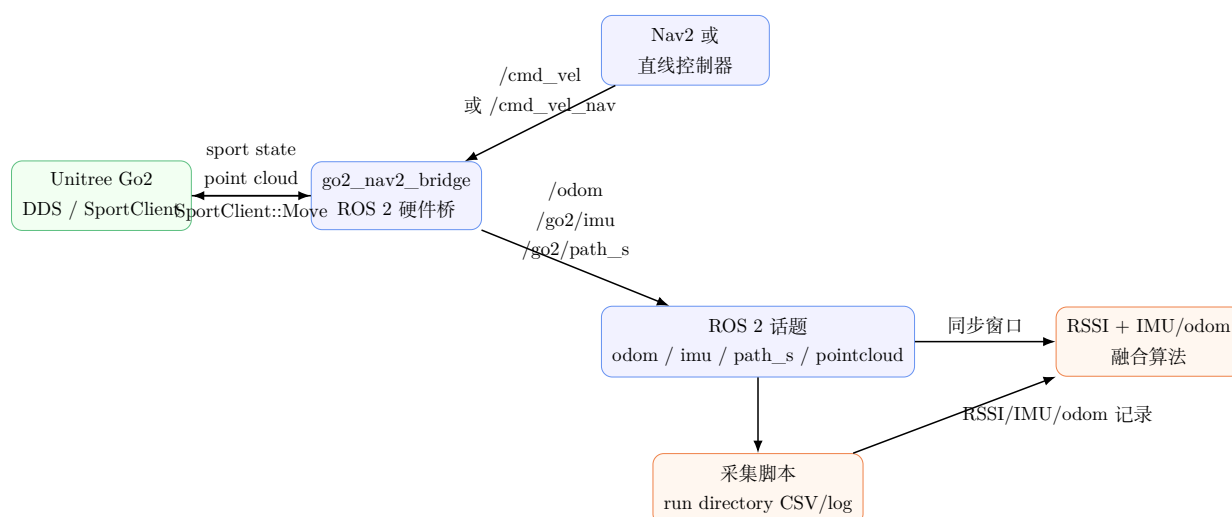


图 1.1: 本项目中的 ROS 2 角色：桥接真实 Go2、组织运动命令、发布标准传感器话题，并为 RSSI/IMU 融合算法提供时间对齐的数据。

从图 1.1 可以看到，本项目里 ROS 2 最核心的价值有三个：

1. **统一接口。** Go2 原始 DDS 数据被转换成 `nav_msgs/msg/Odometry`、`sensor_msgs/msg/Imu`、`sensor_msgs/msg/PointCloud2` 等标准消息。
2. **解耦模块。** 直线控制器、Nav2、诊断脚本、采集脚本都通过话题或 action 接入，不需要直接依赖 Unitree SDK2。
3. **可观测和可调试。** 可以用 `ros2topicecho`、`ros2topichz`、`ros2paramget`、RViz 等工具检查系统。

1.3 本项目当前 ROS 2 文件总表

文件	ROS 2 角色	在本项目中的作用
<code>ros2_ws/src/go2_nav2__bridge/package.xml</code>	package 元数据	声明包名、依赖、运行时组件。ROS 2 用它识别 <code>go2_nav2__bridge</code> 。
<code>ros2_ws/src/go2_nav2__bridge/CMakeLists.txt</code>	构建规则	用 <code>ament_cmake</code> 编译三个 C++ 可执行节点，并链接 Unitree SDK2。
<code>src/go2_nav2__bridge.cpp</code>	ROS 2 node	订阅 Unitree DDS，发布 <code>/odom</code> 、 <code>/go2/imu</code> 、 <code>/go2/path_s</code> 、 <code>/go2/pointcloud</code> ，订阅 <code>/cmd_vel</code> 后调用 <code>SportClient::Move</code> 。
<code>src/go2_direct_straight_controller.cpp</code>	ROS 2 node	直接发布 <code>/cmd_vel</code> ，按 <code>/go2/path_s</code> 、 <code>/odom</code> 或时间判断何时停止。
<code>src/nav2_straight_path_client.cpp</code>	action client	向 Nav2 发送 <code>FollowPath</code> 、 <code>ComputePathToPose</code> 或 <code>NavigateToPose</code> 目标。
<code>launch/go2_nav2_straight.launch.py</code>	launch 文件	一次性启动 Go2 bridge、Nav2 controller、planner、BT navigator、velocity smoother、collision monitor 等节点。
<code>params/go2_nav2__minimal_controller_params.yaml</code>	参数文件	<code>direct/minimal</code> 场景参数，适合无障碍直线 RSSI 采集。
<code>params/go2_nav2_straight_params.yaml</code>	参数文件	<code>obstacle/Nav2</code> 场景参数，打开点云、costmap、velocity smoother 和 collision monitor。
<code>scripts/go2_collect_straight_line_nav2.sh</code>	实验编排脚本	启动 RSSI、IMU、Go2 bridge、direct 或 Nav2 后端，并写入 <code>run directory</code> 。

文件	ROS 2 角色	在本项目中的作用
scripts/go2_diagnose_nav2_perception.sh	诊断脚本	不让机器人运动，只启动感知链路，检查点云、costmap 和 collision monitor 前置条件。

第二章 ROS 2 核心概念：用本项目来理解

2.1 Workspace：工作空间

Workspace

ROS 2 workspace 是一组 package 的开发目录。本项目的 ROS 2 workspace 是 code/ros2_ws。

本项目的结构是：

```
code/  
  ros2_ws/  
    src/  
      go2_nav2_bridge/  
        package.xml  
        CMakeLists.txt  
        launch/  
        params/  
        src/
```

常用构建命令：

```
cd /home/luping/桌面/RSSI/RSSI/code/ros2_ws  
source /opt/ros/humble/setup.bash  
colcon build --packages-select go2_nav2_bridge  
source install/setup.bash
```

为什么一定要 source

source/opt/ros/humble/setup.bash 让 shell 知道 ROS 2 基础环境在哪里；source install/setup.bash 让 shell 知道本项目编译出的 go2_nav2_bridge 在哪里。没有 source，常见现象是 ros2pkgprefixgo2_nav2_bridge 找不到包，或者 ros2run go2_nav2_bridge... 找不到可执行文件。

2.2 Package：功能包

Package

package 是 ROS 2 的最小组织单元。一个 package 可以包含节点源码、launch 文件、参数文件、消息定义、测试和安装规则。

本项目当前的 ROS 2 package 是 go2_nav2_bridge。它的 package.xml 声明了依赖：

```
<depend>rclcpp</depend>
<depend>geometry_msgs</depend>
<depend>nav_msgs</depend>
<depend>sensor_msgs</depend>
<depend>tf2_ros</depend>
<depend>nav2_msgs</depend>
```

这些依赖的含义：

依赖	作用
rclcpp	C++ 节点 API，用来创建 node、publisher、subscriber、timer、parameter。
rclcpp_action	C++ action API，用来和 Nav2 的 FollowPath、NavigateToPose 交互。
geometry_msgs	提供 Twist、PoseStamped、TransformStamped 等几何消息。
nav_msgs	提供 Odometry、Path 等导航消息。
sensor_msgs	提供 Imu、PointCloud2 等传感器消息。
std_msgs	提供 Float32 等简单消息，本项目用它发布 /go2/path_s。
tf2_ros	发布和查询坐标变换，例如 odom->base_link。
nav2_msgs	Nav2 action 和消息类型。

2.3 Node：节点

Node
node 是一个正在运行的 ROS 2 程序。一个节点通常负责一件清晰的事情，例如桥接硬件、发布速度、发送导航目标、诊断点云。

本项目最重要的节点如下：

节点名	来源	职责
go2_nav2_bridge	本项目 C++	硬件桥。把 Go2 状态转换成 ROS 2 话题，把 /cmd_vel 转成 Go2 运动命令。
go2_direct_straight_controller	本项目 C++	直线采集控制器。发固定前进速度，按距离或时间停止。
nav2_straight_path_client	本项目 C++	Nav2 客户端。发送直线路径或目标点给 Nav2。
controller_server	Nav2	根据路径和 costmap 产生局部速度命令。

节点名	来源	职责
velocity_smoother	Nav2	平滑速度命令，限制加速度和减速度。
collision_monitor	Nav2	根据点云安全区修改或阻断速度命令。
go2_nav2_perception_diag	本项目 Python	订阅点云和 costmap，输出感知链路统计。

查看当前节点：

```
ros2 node list
ros2 node info /go2_nav2_bridge
```

2.4 Topic 和 Message：话题与消息

Topic

topic 是连续数据流。publisher 往 topic 发消息，subscriber 从 topic 收消息。topic 适合传感器、速度、状态这类“不断更新”的数据。

Message

message 是 topic 上每一帧数据的结构。例如 /cmd_vel 的消息类型是 geometry_msgs/msg/Twist，里面有线速度和角速度。

本项目的核心 topic：

Topic	消息类型	含义
/cmd_vel	geometry_msgs/msg/Twist	最终发给 Go2 bridge 的速度命令。 bridge 会调用 SportClient::Move(vx,vy,wz)。
/cmd_vel_nav	geometry_msgs/msg/Twist	Nav2 controller 输出的原始速度，之后进入 velocity smoother。
/cmd_vel_smoothed	geometry_msgs/msg/Twist	velocity smoother 输出的平滑速度，之后进入 collision monitor。
/odom	nav_msgs/msg/Odometry	Go2 当前位姿和速度。来源是 Unitree SportModelState。
/tf	tf2_msgs/msg/TFMessage	坐标变换流，本项目主要发布 odom->base_link。
/go2/imu	sensor_msgs/msg/Imu	Go2 IMU 姿态、角速度、线加速度。来源是 SportModelState.imu_state()。

Topic	消息类型	含义
/go2/path_s	std_msgs/msg/Float32	bridge 根据 Go2 position 增量累计出的路径进度。
/go2/pointcloud	sensor_msgs/msg/PointCloud2	Go2 点云，供 Nav2 costmap 和 collision monitor 使用。
/local_costmap/costmap	nav_msgs/msg/OccupancyGrid	Nav2 局部代价地图，诊断脚本会读取它。

常用命令：

```
ros2 topic list
ros2 topic info /odom
ros2 topic echo /go2/path_s
ros2 topic hz /go2/imu
ros2 interface show geometry_msgs/msg/Twist
```

2.5 Publisher 和 Subscriber：发布与订阅

在 go2_nav2_bridge.cpp 里，bridge 创建 publisher：

```
odom_pub_ = create_publisher<nav_msgs::msg::Odometry>(odom_topic_, rclcpp::SensorDataQoS());
imu_pub_ = create_publisher<sensor_msgs::msg::Imu>(imu_topic_, rclcpp::SensorDataQoS());
path_s_pub_ = create_publisher<std_msgs::msg::Float32>(path_s_topic_, 10);
point_cloud_pub_ = create_publisher<sensor_msgs::msg::PointCloud2>(
    ros_point_cloud_topic_, rclcpp::SensorDataQoS());
```

也创建 subscriber：

```
cmd_vel_sub_ = create_subscription<geometry_msgs::msg::Twist>(
    cmd_vel_topic_, 10,
    std::bind(&Go2Nav2Bridge::onTwist, this, std::placeholders::_1));
```

直白理解：

- publisher 是“往外广播数据”。
- subscriber 是“监听别人发来的数据”。
- bridge 既是 publisher，也是 subscriber，因为它既发布 Go2 状态，也接收运动命令。

2.6 Parameter: 参数

Parameter

parameter 是节点启动时或运行时可配置的变量。它让同一个程序在不同实验场景下使用不同配置，而不需要改 C++ 源码。

go2_nav2_bridge.cpp 中有大量 declare_parameter:

```
network_interface_ = declare_parameter<std::string>("network_interface", "");
cmd_vel_topic_ = declare_parameter<std::string>("cmd_vel_topic", "/cmd_vel");
odom_topic_ = declare_parameter<std::string>("odom_topic", "/odom");
imu_topic_ = declare_parameter<std::string>("imu_topic", "/go2/imu");
publish_point_cloud_ = declare_parameter<bool>("publish_point_cloud", true);
cmd_timeout_sec_ = declare_parameter<double>("cmd_timeout_sec", 0.5);
max_vx_mps_ = declare_parameter<double>("max_vx_mps", 0.35);
```

这些参数在 YAML 中配置:

```
go2_nav2_bridge:
  ros__parameters:
    network_interface: eth0
    cmd_vel_topic: /cmd_vel
    odom_topic: /odom
    imu_topic: /go2/imu
    publish_point_cloud: true
    cmd_timeout_sec: 0.6
    max_vx_mps: 0.12
```

查询参数:

```
ros2 param list /go2_nav2_bridge
ros2 param get /go2_nav2_bridge max_vx_mps
```

2.7 Launch: 一键启动多个节点

Launch

launch 文件是 ROS 2 的启动编排。它能一次启动多个节点，并给它们传参数、重映射话题、设置条件。

本项目的 go2_nav2_straight.launch.py 做了这些事:

- 永远启动 go2_nav2_bridge。
- 当 move_backend=nav2 时启动 Nav2 controller、velocity smoother、collision monitor。
- 当 use_planner=true 或 use_bt_navigator=true 时启动 planner。
- 当 use_bt_navigator=true 时启动 behavior server 和 BT navigator。

- 启动 lifecycle manager，使 Nav2 管理节点进入 active 状态。

启动示例：

```
ros2 launch go2_nav2_bridge go2_nav2_straight.launch.py \
  params_file:=/path/to/go2_nav2_straight_params.yaml \
  move_backend:=nav2 \
  use_bt_navigator:=false \
  use_planner:=false \
  network_interface:=enp5s0
```

2.8 Remapping：话题重映射

Nav2 controller 默认输出 cmd_vel。本项目在 launch 里把它重映射成 /cmd_vel_nav：

```
Node(
  package="nav2_controller",
  executable="controller_server",
  name="controller_server",
  remappings=[("cmd_vel", "/cmd_vel_nav")],
)
```

这样做的原因是要把速度链路拆开：

```
/cmd_vel_nav → velocity_smoother → /cmd_vel_smoothed → collision_monitor → /cmd_vel
→ go2_nav2_bridge
```

也就是说，/cmd_vel 是真正进入 Go2 bridge 的最终速度命令。

2.9 TF：坐标变换

TF

TF 是 ROS 2 中维护坐标系关系的系统。机器人导航至少需要知道 odom 坐标系和 base_link 坐标系之间的关系。

本项目 bridge 发布：

```
odom->base_link
```

go2_nav2_bridge.cpp 中相关逻辑是：

```
tf_msg.header.frame_id = odom_frame_;
tf_msg.child_frame_id = base_frame_;
tf_msg.transform.translation.x = odom.pose.pose.position.x;
tf_msg.transform.translation.y = odom.pose.pose.position.y;
tf_msg.transform.rotation = odom.pose.pose.orientation;
tf_broadcaster->sendTransform(tf_msg);
```

检查 TF：

```
ros2 run tf2_ros tf2_echo odom base_link
```

TF 出问题时会怎样

Nav2 controller、planner、costmap 都依赖 TF。如果 odom->base_link 不存在、时间戳太旧、frame 名不一致，常见现象是 Nav2 active 了但不出速度，或者日志里出现 transform timeout。

2.10 QoS：通信质量配置

QoS

QoS 决定 ROS 2 通信的可靠性、缓存深度、是否保留上一帧等策略。传感器数据通常用低延迟策略，地图类数据常用可靠和 transient local 策略。

本项目中：

- /odom、/go2/imu、/go2/pointcloud 使用 rclcpp::SensorDataQoS()，适合高频传感器。
- /go2/path_s 使用普通 depth 10，因为它是轻量路径进度值。
- 诊断脚本订阅 /go2/pointcloud 时使用 best effort，订阅 /local_costmap/costmap 时使用 reliable + transient local。

如果 ros2topic echo 看不到某些传感器数据，但节点确实在发布，可能是 QoS 不匹配。可用：

```
ros2 topic info /go2/pointcloud --verbose
```

2.11 Action：长任务接口

Action

action 用于“会持续一段时间、需要反馈、可以取消”的任务。例如导航到目标点、跟随路径。

本项目的 nav2_straight_path_client 使用三个 Nav2 action：

Action	本项目用途
FollowPath	给 Nav2 controller 一条路径，让 controller 产生局部速度。
ComputePathToPose	请求 planner 从当前位置规划到目标点。
NavigateToPose	交给 BT navigator 完成目标导航，可包含重规划和行为树逻辑。

初学时可以这样区分：

- topic 像“直播数据流”：IMU、odom、cmd_vel。

- service 像 “一问一答”：请求一次，返回一次。
- action 像 “执行任务”：发送目标，期间有反馈，最后有结果，还可以取消。

2.12 Lifecycle: Nav2 管理节点状态

Nav2 很多节点不是普通 node，而是 lifecycle node。它们有状态：unconfigured、inactive、active、finalized。只有 active 后才真正工作。

本项目 launch 使用 lifecycle_manager 自动激活 Nav2 节点。采集脚本里也会等待日志出现：

```
Managed nodes are active
```

命令行检查：

```
ros2 lifecycle get /controller_server
```

第三章 本项目 ROS 2 Package 逐文件对照

3.1 package.xml: 告诉 ROS 2 这个包是什么

package.xml 里最关键的是三类信息:

- 包名: go2_nav2_bridge。
- 构建工具: ament_cmake。
- 编译和运行依赖: rclcpp、nav_msgs、sensor_msgs、tf2_ros、nav2_msgs、Nav2 相关包。

本项目还声明了 Nav2 运行依赖:

```
<exec_depend>nav2_controller</exec_depend>
<exec_depend>nav2_costmap_2d</exec_depend>
<exec_depend>nav2_velocity_smoother</exec_depend>
<exec_depend>nav2_collision_monitor</exec_depend>
<exec_depend>nav2_lifecycle_manager</exec_depend>
```

这表示: 编译本项目 C++ 代码不一定需要直接链接这些 Nav2 进程, 但运行 launch 时需要它们存在。

3.2 CMakeLists.txt: 告诉 colcon 怎么编译

CMakeLists.txt 中定义了三个可执行节点:

```
add_executable(go2_nav2_bridge src/go2_nav2_bridge.cpp)
add_executable(nav2_straight_path_client src/nav2_straight_path_client.cpp)
add_executable(go2_direct_straight_controller src/go2_direct_straight_controller.cpp)
```

然后安装:

```
install(TARGETS
  go2_nav2_bridge
  nav2_straight_path_client
  go2_direct_straight_controller
  DESTINATION lib/${PROJECT_NAME})

install(DIRECTORY launch params
  DESTINATION share/${PROJECT_NAME})
```

这两段决定了：

- `ros2run go2_nav2_bridge go2_nav2_bridge` 可以找到可执行文件。
- `ros2launch go2_nav2_bridge go2_nav2_straight.launch.py` 可以找到 launch 文件。
- `ros2pkgprefix go2_nav2_bridge` 下可以找到安装后的 params 文件。

3.3 Unitree SDK2 链接

本项目 bridge 需要链接 Unitree SDK2：

```
find_library(UNITREE_SDK2_LIB
  NAMES unitree_sdk2
  PATHS "${UNITREE_SDK2_DIR}/lib/${UNITREE_SDK2_ARCH}"
  NO_DEFAULT_PATH)
```

如果 SDK2 不在默认位置，可以这样构建：

```
colcon build --packages-select go2_nav2_bridge \
  --cmake-args -DUNITREE_SDK2_DIR=/path/to/unitree_sdk2
```

运行时还要保证动态库路径正确。采集脚本会把：

```
code/unitree_sdk2/thirdparty/lib/$(uname -m)
```

加入 `LD_LIBRARY_PATH`。

第四章 Go2 Bridge: 本项目最关键的 ROS 2 节点

4.1 Bridge 的输入输出

go2_nav2_bridge 是本项目 ROS 2 链路的中心。它一边连接 Unitree DDS/SportClient, 一边连接 ROS 2。

方向	接口	说明
Go2 → ROS 2	rt/ sportmodestate	读取 Go2 position、velocity、rpy、gyro、acc, 转换成 odom、imu、path_s、TF。
Go2 → ROS 2	rt/utlidar/cloud	读取 Unitree 点云, 转换成 sensor_msgs/msg/PointCloud2, 可做外参变换和自过滤。
ROS 2 → Go2	/cmd_vel	接收线速度和角速度, 限幅后调用 SportClient::Move。

4.2 速度命令如何进入 Go2

当 bridge 收到 /cmd_vel:

```
void onTwist(const geometry_msgs::msg::Twist::SharedPtr msg)
{
    sendVelocity(msg->linear.x, msg->linear.y, msg->angular.z);
}
```

sendVelocity 会做三件重要的事:

1. 把非有限值变成 0, 避免 NaN/inf 进入机器人。
2. 用 max_vx_mps、max_vy_mps、max_wz_radps 对速度限幅。
3. 调用 sport_client_->Move(vx,vy,wz)。

简化逻辑:

```
safe_vx = clamp(vx, -max_vx_mps_, max_vx_mps_);
safe_vy = clamp(vy, -max_vy_mps_, max_vy_mps_);
```

```
safe_wz = clamp(wz, -max_wz_radps_, max_wz_radps_);
sport_client_->Move(safe_vx, safe_vy, safe_wz);
```

速度限幅是安全边界

Nav2、直线控制器、命令行都可能发 `/cmd_vel`。真正进入 Go2 前，bridge 的 `max_vx_mps`、`max_vy_mps`、`max_wz_radps` 是最后一道软件限幅。实验时不要只看 controller 参数，也要看 bridge 参数。

4.3 watchdog: 为什么机器人会自动停

bridge 中有 watchdog timer。若持续一段时间没有新的非零速度命令，就发送停止：

```
if (age > cmd_timeout_sec_) {
    sendStop();
}
```

这对应参数：

```
cmd_timeout_sec: 0.6
control_watchdog_hz: 20.0
```

直白理解：ROS 2 速度命令必须持续发布。如果上游节点崩了、网络断了、Nav2 卡了，bridge 不会让 Go2 一直按旧速度走。

4.4 odom 是怎么来的

`/odom` 不是 RSSI 算出来的，也不是 Nav2 算出来的。本项目的 `/odom` 来自 Go2 的 `SportModeState`：

```
const auto & position = state.position();
const auto & velocity = state.velocity();
const auto & rpy = state.imu_state().rpy();
```

bridge 把这些字段填入 `nav_msgs/msg/Odometry`：

```
odom.pose.pose.position.x = position[0];
odom.pose.pose.position.y = position[1];
odom.pose.pose.position.z = position[2];
odom.twist.twist.linear.x = velocity[0];
odom.twist.twist.linear.y = velocity[1];
odom.twist.twist.angular.z = state.yaw_speed();
```

所以本项目里的 odom 可以理解为：

Go2 底层状态估计给出的短期位姿和速度，bridge 把它包装成标准 ROS 2 odometry。

它适合做短时连续性、运动约束、Nav2 controller 输入和实验参考；不应把它当成 RSSI 研究算法的主定位结果。

4.5 /go2/path_s 是怎么来的

bridge 每次收到 Go2 sport state, 会比较当前 position 和上一帧 position:

```
const double dx = position[0] - last_x_;
const double dy = position[1] - last_y_;
const double ds = std::hypot(dx, dy);
path_s_m_ += ds;
```

然后发布:

```
std_msgs::msg::Float32 path_s;
path_s.data = static_cast<float>(path_s_m_);
path_s_pub_->publish(path_s);
```

/go2/path_s 的含义是“从本次 bridge 启动后累计走过的路径长度”。它不是全局地图上的真实 s , 但在直线采集时很有用, 因为它可以告诉直线控制器“这一趟已经走了多少米”。

4.6 IMU 是怎么发布的

Go2 sport state 中的 IMU 字段被转成标准 sensor_msgs/msg/Imu:

```
imu.orientation = odom.pose.pose.orientation;
imu.angular_velocity.x = gyro[0];
imu.angular_velocity.y = gyro[1];
imu.angular_velocity.z = gyro[2];
imu.linear_acceleration.x = acc[0];
imu.linear_acceleration.y = acc[1];
imu.linear_acceleration.z = acc[2];
imu_pub_->publish(imu);
```

融合算法使用 IMU/odom 的方式可以简单理解为: RSSI 给当前位置观测, IMU/odom 给短时间内“应该移动了多少”的运动约束, 最后融合出预测 s 。

4.7 点云处理: 外参、过滤和新鲜度检查

当 publish_point_cloud=true 时, bridge 会把 Unitree 点云发布成 /go2/pointcloud。中间会做三件事:

1. 根据 point_cloud_xyz_offset 和 point_cloud_rpy_offset 做坐标变换。
2. 根据 self_filter_boxes 删除机器人自身附近的点。
3. 更新点云时间戳, 给 require_fresh_point_cloud_for_motion 使用。

如果打开:

```
require_fresh_point_cloud_for_motion: true  
point_cloud_motion_timeout_sec: 0.8
```

那么点云太久没更新时, bridge 会阻止非零速度命令。这适合 obstacle/Nav2 测试, 但不适合没有点云的 minimal 直线采集。

第五章 直线控制器与 Nav2 客户端

5.1 直线控制器：最小闭环

`go2_direct_straight_controller` 是本项目推荐的无障碍直线采集后端。它不依赖 `Nav2 controller`，只做简单稳定的事情：

1. 订阅 `/go2/path_s` 和 `/odom`。
2. 按 `speed_mps` 发布 `/cmd_vel`。
3. 根据 `distance_source` 判断是否到达目标距离。
4. 到达后发布 0 速度并退出。

核心参数：

参数	常用值	含义
<code>distance_m</code>	1.0	本次直线采集目标距离。
<code>speed_mps</code>	0.20	前进速度。源码中会限制在 0.03--0.35。
<code>max_runtime_sec</code>	30.0	超时保护，避免一直走。
<code>control_hz</code>	20.0	发布速度命令频率。
<code>distance_source</code>	<code>path_s</code>	停止依据，可选 <code>path_s</code> 、 <code>odom</code> 、 <code>time</code> 。
<code>cmd_vel_topic</code>	<code>/cmd_vel</code>	输出速度话题。

运行示例：

```
ros2 run go2_nav2_bridge go2_direct_straight_controller \  
  --ros-args \  
  --params-file /path/to/go2_nav2_minimal_controller_params.yaml \  
  -p distance_m:=1.0 \  
  -p speed_mps:=0.20 \  
  -p distance_source:=path_s
```

5.2 Nav2 客户端：把目标交给 Nav2

`nav2_straight_path_client` 是本项目的 Nav2 action 客户端。它的工作流程是：

- 1. 通过 TF 查询当前 odom->base_link。
- 2. 根据当前位置生成前方 distance_m 的目标或路径。
- 3. 如果 use_planner=true，请求 planner 生成路径。
- 4. 如果 use_navigate_to_pose=true，走 BT navigator。
- 5. 否则直接发送 FollowPath。

常用参数：

参数	常用值	含义
distance_m	0.5--1.0	目标距离。
path_resolution_m	0.10	自生成直线路径的点间距。
odom_frame / base_frame	odom / base_link	查询 TF 的坐标系。
controller_id	FollowPath	controller server 中的插件 ID。
action_name	follow_path	FollowPath action 名称。
use_planner	false 或 true	是否先请求 ComputePathToPose。
use_navigate_to_pose	false 或 true	是否使用 BT navigator。
finish_on_path_s	false	是否按 /go2/path_s 到达距离后取消 action。

第六章 Launch 与参数文件：本项目怎样一键启动

6.1 Launch 参数

go2_nav2_straight.launch.py 提供这些入口：

Launch 参数	默认值	含义
params_file	straight params	传给 bridge/Nav2/控制器的 YAML 文件。
network_interface	eth0	Unitree SDK2 DDS 使用的网卡。外部电脑常见是 enp5s0。
move_backend	nav2	nav2 启动 Nav2 控制链；cmd_vel 只启动 bridge；采集脚本还支持 direct。
use_bt_navigator	false	是否启动 BT navigator 和 behavior server。
use_planner	false	是否启动 planner server。

6.2 minimal 参数文件

go2_nav2_minimal_controller_params.yaml 适合无障碍直线采集。特点是：

- publish_point_cloud=false，不依赖点云。
- max_vy_mps=0.0、max_wz_radps=0.0，限制侧向和旋转。
- local costmap 只有 inflation layer，不以点云避障为主。
- direct controller 使用 distance_source=path_s。

它更像“稳定采 RSSI/IMU 数据”的配置，而不是完整避障导航配置。

6.3 straight/obstacle 参数文件

go2_nav2_straight_params.yaml 适合 Nav2 obstacle 测试。特点是：

- publish_point_cloud=true，发布 /go2/pointcloud。
- local/global costmap 使用 voxel_layer+inflation_layer。
- controller 使用 DWB，并允许小幅 vy 和 wz。

- velocity smoother 限制速度变化。
- collision monitor 设置 FrontStop、SideStop、BodySlow、SideSlow 区域。
- require_fresh_point_cloud_for_motion=true, 点云不新鲜时阻断运动。

6.4 采集脚本如何编排 ROS 2

scripts/go2_collect_straight_line_nav2.sh 是本项目的一键采集入口。它做的事情比普通 ROS 2 launch 多：

1. 检查是否已有 Go2/Nav2 进程残留。
2. source ROS 2 和本项目 overlay。
3. 检查 go2_nav2_bridge 是否已经 build/source。
4. 根据 MOVE_BACKEND 选择参数文件。
5. 启动 RSSI 采集、IMU logger、Go2/Nav2 launch。
6. 根据 backend 运行 direct controller、Nav2 client 或简单 ros2topicpub。
7. 停止后台进程，复制远程 RSSI 文件，写入 run_metadata.txt。

推荐直线采集命令：

```
cd /home/luping/桌面/RSSI/RSSI/code

RUN_DIR="$HOME/go2_rssi_runs/direct_1m_rssi" \
MOVE_BACKEND=direct \
RSSI_MODE=remote \
ROBOT_SSH=unitree@192.168.123.18 \
ROBOT_PROJECT_DIR=/home/unitree/rssi_go2 \
ROBOT_NETWORK_INTERFACE=eth0 \
BT_TTY=/dev/ttyACM1 \
DIRECT_DISTANCE_SOURCE=path_s \
MAX_RUNTIME_SEC=25 \
scripts/go2_collect_straight_line_nav2.sh enp5s0 1.0 0.20 1.0 0.5
```

输出目录通常包含：

- rssi_realtime_windows.csv
- rssi_raw_stream.csv
- imu_stream.csv 或 remote_imu_stream.csv
- nav2_launch.log
- run_metadata.txt

第七章 构建、运行和调试

7.1 从零构建

外部电脑推荐 Ubuntu 22.04 + ROS 2 Humble:

```
cd /home/luping/桌面/RSSI/RSSI/code/ros2_ws
source /opt/ros/humble/setup.bash
colcon build --packages-select go2_nav2_bridge
source install/setup.bash
```

检查:

```
ros2 pkg prefix go2_nav2_bridge
ros2 pkg executables go2_nav2_bridge
```

应该能看到:

- go2_nav2_bridge
- go2_direct_straight_controller
- nav2_straight_path_client

7.2 只启动 bridge

用于检查 Go2 DDS、odom、IMU、path_s:

```
ros2 run go2_nav2_bridge go2_nav2_bridge \
  --ros-args \
  --params-file /path/to/go2_nav2_minimal_controller_params.yaml \
  -p network_interface:=enp5s0
```

另一个终端检查:

```
ros2 topic hz /odom
ros2 topic hz /go2/imu
ros2 topic echo /go2/path_s
ros2 run tf2_ros tf2_echo odom base_link
```

7.3 手动发速度命令

只在安全空旷环境做短时间测试：

```
ros2 topic pub --rate 10 /cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.05, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}" \
  --times 20

ros2 topic pub --once /cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

7.4 直线采集推荐路径

推荐优先使用：

```
MOVE_BACKEND=direct RSSI_MODE=off MAX_RUNTIME_SEC=20 \
scripts/go2_collect_straight_line_nav2.sh enp5s0 1.0 0.20 1.0 0.5
```

确认稳定后再打开 RSSI：

```
MOVE_BACKEND=direct RSSI_MODE=remote \
ROBOT_SSH=unitree@192.168.123.18 \
scripts/go2_collect_straight_line_nav2.sh enp5s0 1.0 0.20 1.0 0.5
```

7.5 Nav2 感知链路诊断

在 obstacle/Nav2 测试前，先不运动，只检查点云和 costmap：

```
RUN_DIR="$HOME/go2_rssi_runs/nav2_perception_diag" \
scripts/go2_diagnose_nav2_perception.sh enp5s0 12
```

诊断脚本会订阅：

- /go2/pointcloud
- /local_costmap/costmap

并统计 front stop、side stop、body slow 等区域内点云数量。若点云 frame、外参或自过滤不对，先修这些，不要直接做运动测试。

7.6 常用排查命令

命令	用途
ros2nodelist	看节点是否启动。

命令	用途
ros2nodeinfo/go2_nav2_bridge	看 bridge 发布/订阅了哪些话题。
ros2topiclist	看当前所有 topic。
ros2topichz/go2/imu	看 IMU 是否持续发布。
ros2topicecho/go2/path_s	看累计路径进度是否变化。
ros2topicecho/cmd_vel	看最终进入 bridge 的速度命令。
ros2paramget/go2_nav2_bridg max_vx_mps	查 bridge 速度限幅。
ros2lifecycleget/controller_server	查 Nav2 controller 是否 active。
ros2run tf2_ros tf2_echo odom_base_link	查 TF 是否可用。
tail -n 220 "\$RUN_DIR/nav2_launch.log"	查采集 run 的 ROS 2/Nav2 日志。

第八章 ROS 2 和 RSSI/IMU 融合算法的关系

8.1 ROS 2 不直接等于融合算法

本项目中，ROS 2 是数据和控制基础设施；RSSI/IMU 融合算法是定位研究主线。二者关系如下：

- ROS 2 bridge 提供 /odom、/go2/imu、/go2/path_s。
- RSSI 采集程序提供 rssi_realtime_windows.csv。
- 采集脚本把 RSSI、IMU、ROS 2 日志放到同一个 run directory。
- 融合算法按时间戳对齐 RSSI window 与 IMU/odom，输出预测路径进度 $\hat{s}$ 。
- Nav2 localization 输出不作为 RSSI 主算法输入，可以作为调试或对照参考。

8.2 一条数据怎样进入训练或评估

一次直线采集大致产生：

```
run_dir/  
  rssi_realtime_windows.csv  
  rssi_raw_stream.csv  
  imu_stream.csv  
  remote_imu_stream.csv  
  nav2_launch.log  
  run_metadata.txt
```

融合算法使用它们时，直白流程是：

1. RSSI window 给出当前无线指纹。
2. IMU/odom 给出这个窗口附近的运动趋势和短时位移。
3. 真实采集路线定义全局中心线 s 。
4. 训练时把 RSSI/IMU/odom 特征和真实 s 对齐。
5. 在线时 RSSI 先猜位置，IMU/odom 限制跳变和短时连续性，最后输出 $\hat{s}$ 。

第九章 常见问题

9.1 ros2 run 找不到节点

检查三件事：

```
source /opt/ros/humble/setup.bash
source /home/luping/桌面/RSSI/RSSI/code/ros2_ws/install/setup.bash
ros2 pkg executables go2_nav2_bridge
```

如果没有 executables，重新 build。

9.2 /odom 和 /go2/imu 没有数据

可能原因：

- network_interface 错了。
- Unitree DDS 网络没有连通。
- Unitree SDK2 动态库路径没有加入 LD_LIBRARY_PATH。
- bridge 没启动或启动后报错退出。

优先看：

```
tail -n 120 "$RUN_DIR/nav2_launch.log"
ros2 node list
ros2 topic list
```

9.3 Nav2 active 了但机器人不动

按顺序查：

1. ros2topicecho/cmd_vel_nav: Nav2 controller 有没有输出。
2. ros2topicecho/cmd_vel_smoothed: velocity smoother 有没有输出。
3. ros2topicecho/cmd_vel: collision monitor 后还有没有最终速度。
4. ros2paramget/go2_nav2_bridgerequire_fresh_point_cloud_for_motion: 是否要求新鲜点云。
5. ros2run tf2_rostf2_echo odombase_link: TF 是否正常。

9.4 点云导致机器人一直停

如果 obstacle 参数中开启 require_fresh_point_cloud_for_motion=true，点云不更新或 frame 不对会阻断非零速度。先运行：

```
scripts/go2_diagnose_nav2_perception.sh enp5s0 12
```

确认 /go2/pointcloud、/local_costmap/costmap、collision monitor 区域统计正常后，再做运动测试。

9.5 什么时候用 direct，什么时候用 Nav2

模式	适合场景
MOVE_BACKEND=direct	推荐用于 RSSI/IMU 直线采集。链路短、变量少、结果更易复现。
MOVE_BACKEND=cmd_vel	最简单的手动/脚本速度测试，只适合短距离安全检查。
MOVE_BACKEND=nav2	用于 obstacle、local planning、collision monitor 对照实验。调试成本更高，必须先确认点云和 TF 正常。

第十章 学习路线

作为 ROS 2 新手，建议用本项目按以下顺序练习：

1. 只 build `go2_nav2_bridge`，确认 `ros2pkgexecutables` 能看到三个节点。
2. 只启动 `go2_nav2_bridge`，观察 `/odom`、`/go2/imu`、`/go2/path_s`。
3. 用 `ros2topicpub` 发很短的 `/cmd_vel`，确认 `bridge` 能控制 Go2。
4. 运行 `go2_direct_straight_controller`，理解 topic 闭环。
5. 运行 `go2_collect_straight_line_nav2.sh`，理解 run directory 和采集文件。
6. 运行 `go2_diagnose_nav2_perception.sh`，理解点云、costmap、QoS 和 TF。
7. 最后再进入 Nav2 FollowPath、planner、BT navigator。

最重要的心智模型

ROS 2 不是一个单独的大程序，而是一组节点通过标准接口组合起来。本项目里最稳的理解方式是沿着数据流看：Go2 DDS 进入 `bridge`，`bridge` 发布 ROS 2 标准话题，控制器或 Nav2 发布速度，`bridge` 把速度交给 Go2，采集脚本把 RSSI/IMU/odom/log 保存给融合算法使用。